

Key Software Metrics

Measuring success...

Contents

- North Star
- Agile Scrum Metrics / KPIs
- Harmful Metrics
- GIT Analysis

North Star

North Star Framework



North Star Metric



North Star Framework

- Process/framework to identify
 - single
 - critical metric

“best captures the core value that your product delivers to [its] customers”



Sean Ellis

Elements of North Star Framework

4

“The Work”

3

Input Metrics”

Input 1

Input 2

Input 3

Input...

1

North Star Metric

2

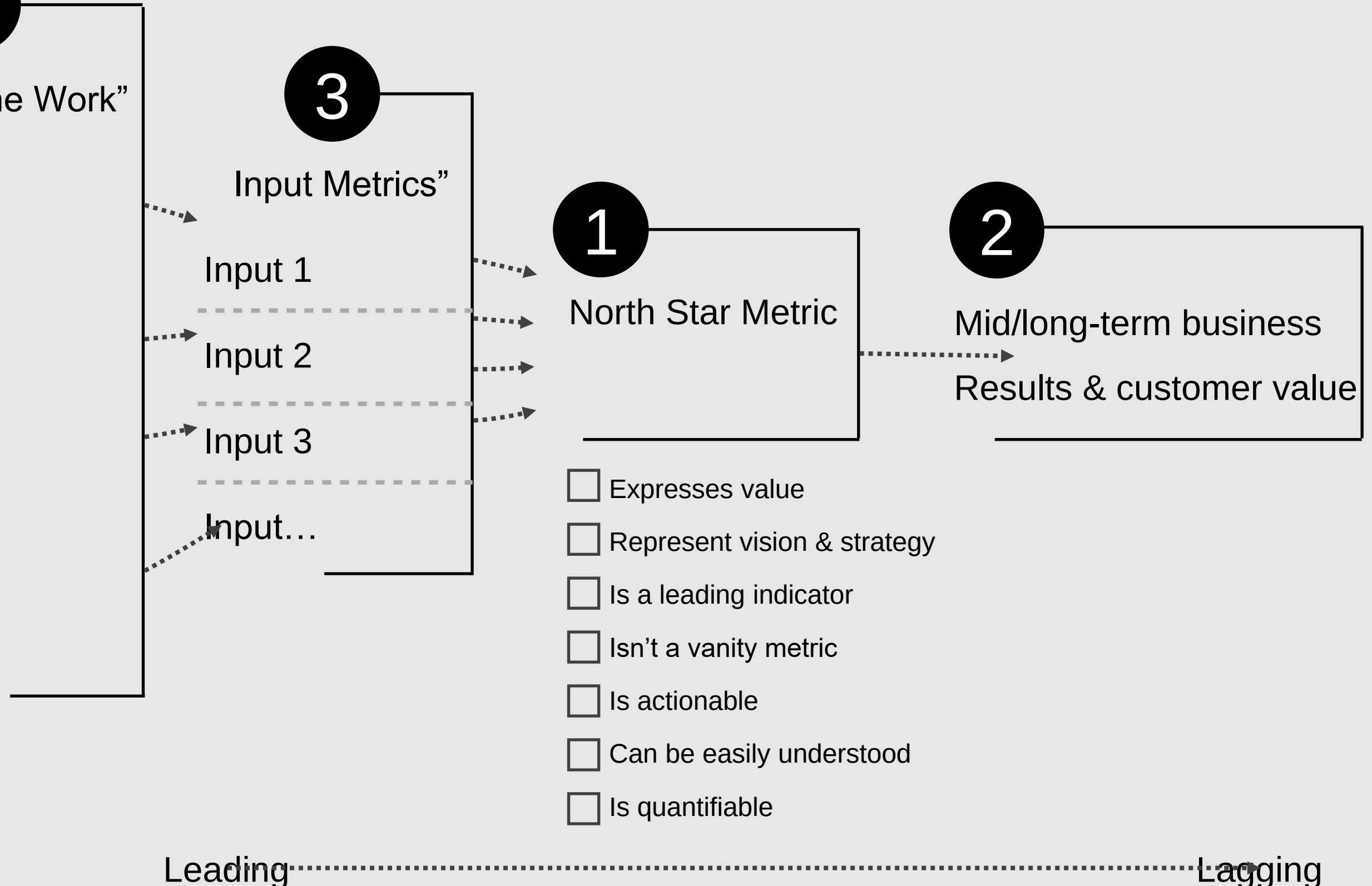
Mid/long-term business

Results & customer value

- ☐ Expresses value
- ☐ Represent vision & strategy
- ☐ Is a leading indicator
- ☐ Isn't a vanity metric
- ☐ Is actionable
- ☐ Can be easily understood
- ☐ Is quantifiable

Leading

Lagging



Qualities of a good North Star

- **Expresses Value** - it should be clear why it conveys value to the user.
- **Describes product vision and Strategy** - Once discovered a strong NSM connects to your product strategy and vision.
- **Serve as a leading indicator** - Predict future outcomes rather than previous events.
- **Actionable** - you should be able to influence strong NSM through work and action.
- **Comprehensible** - understood by non-technical audience.
- **Measurable** - must be able to track it.

Example 1 - Spotify



“Time spent listening to music by subscribers”

Example 2 - Facebook



Facebook

“Monthly Active Users”

Example 2 - Burger King



Digital Team

“Digital transactions per user”

School of Computing NSM

- What would be a strong NSM for School of Computing?

School of Computing - NSM



% of students who are in
graduate jobs with annual
salary > 30k



% of first class or
2:1 students



Student
satisfaction



Placement rate



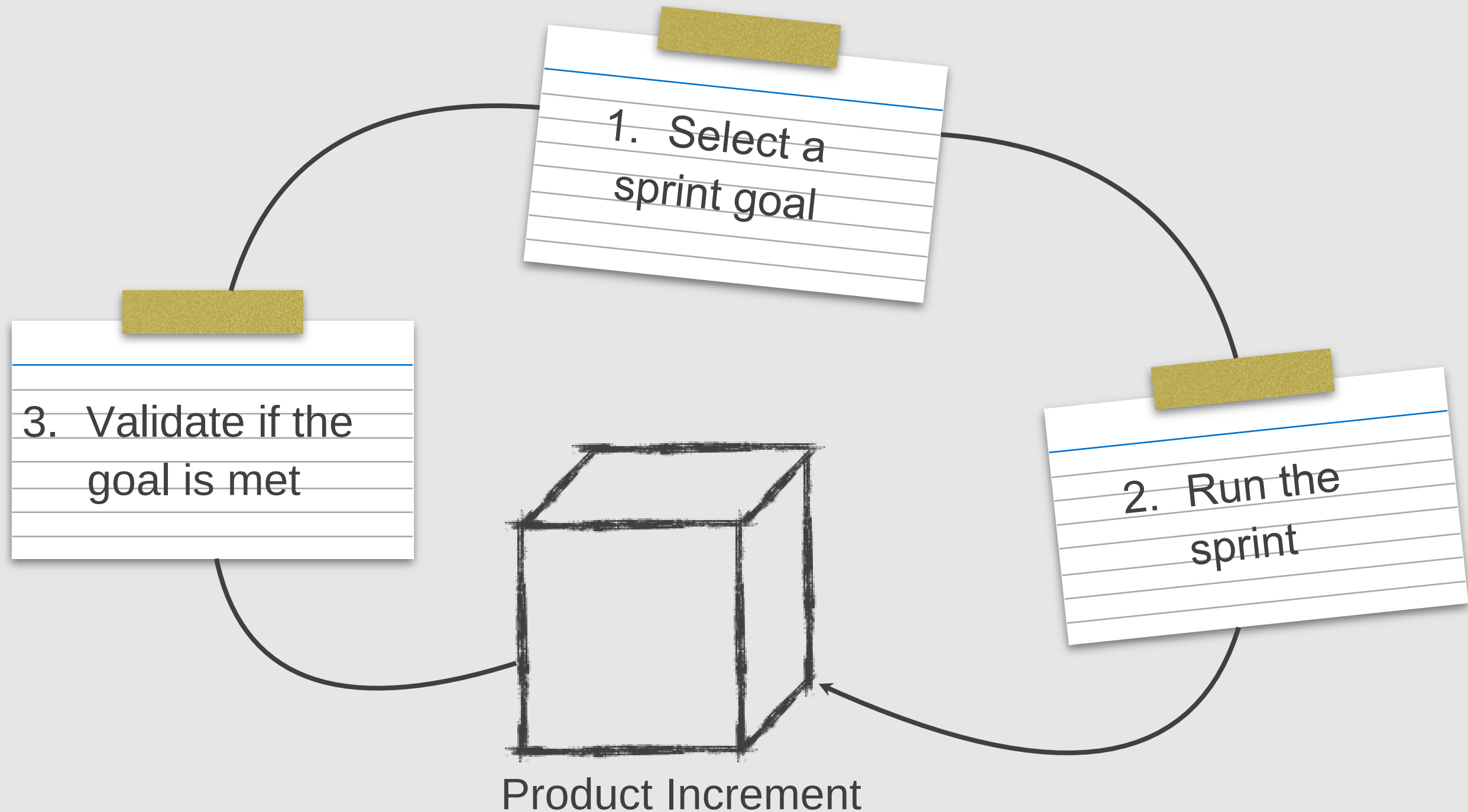
Scholarships rate



Hackathons rate

Agile Scrum Metrics / KPIs

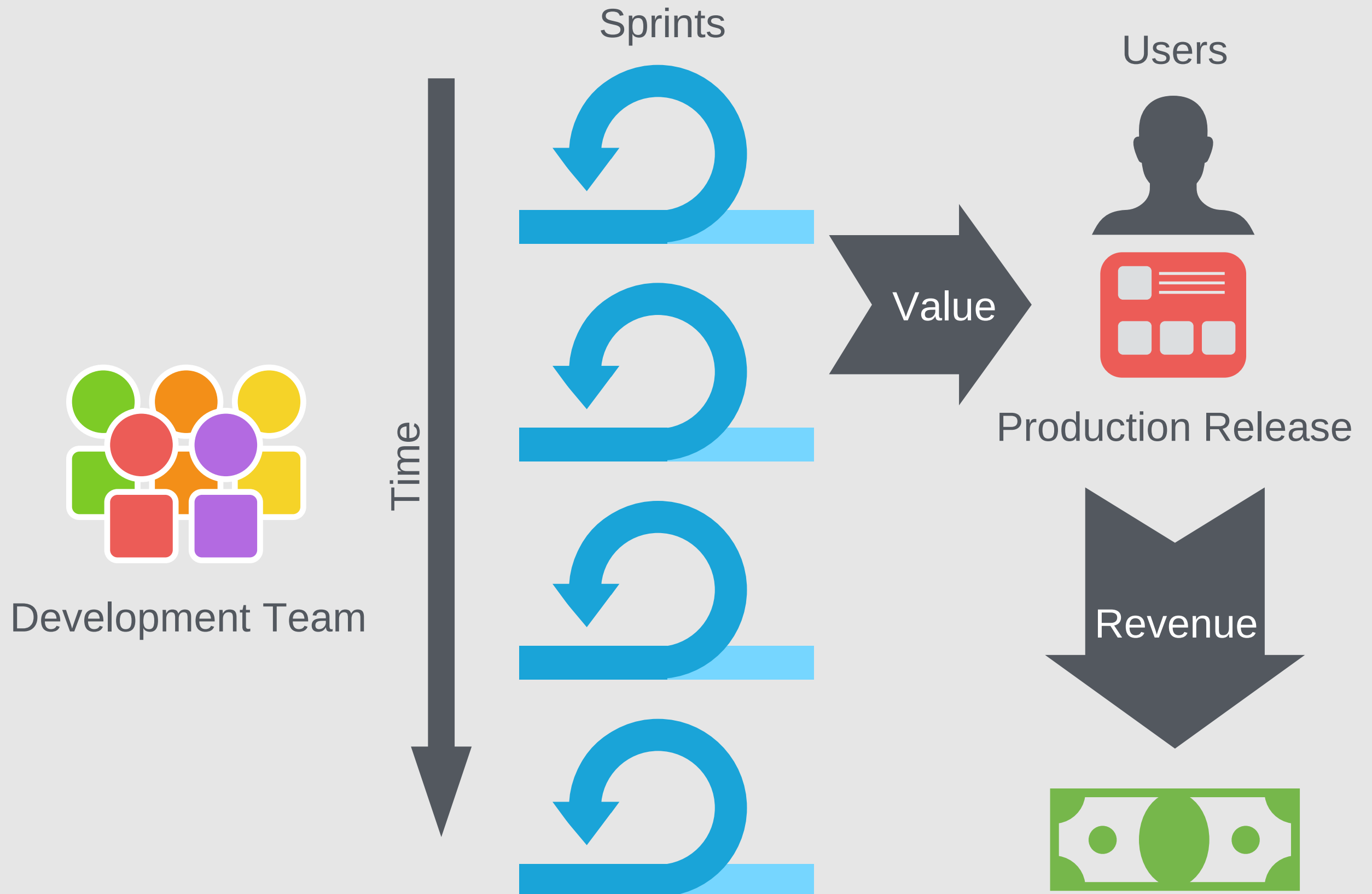
Sprint Goal Success



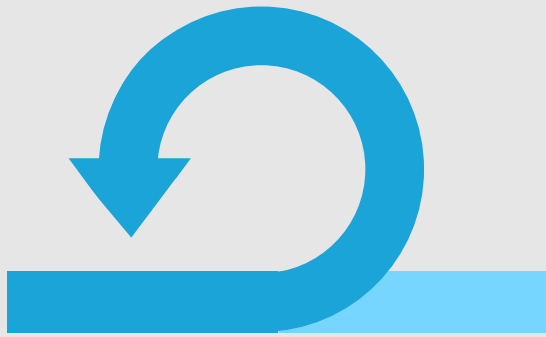
Measuring Business Goals

- Time to market
- ROI
- Capital redeployment
- Customer satisfaction

Time to market



ROI



Cost to run Sprints



VS



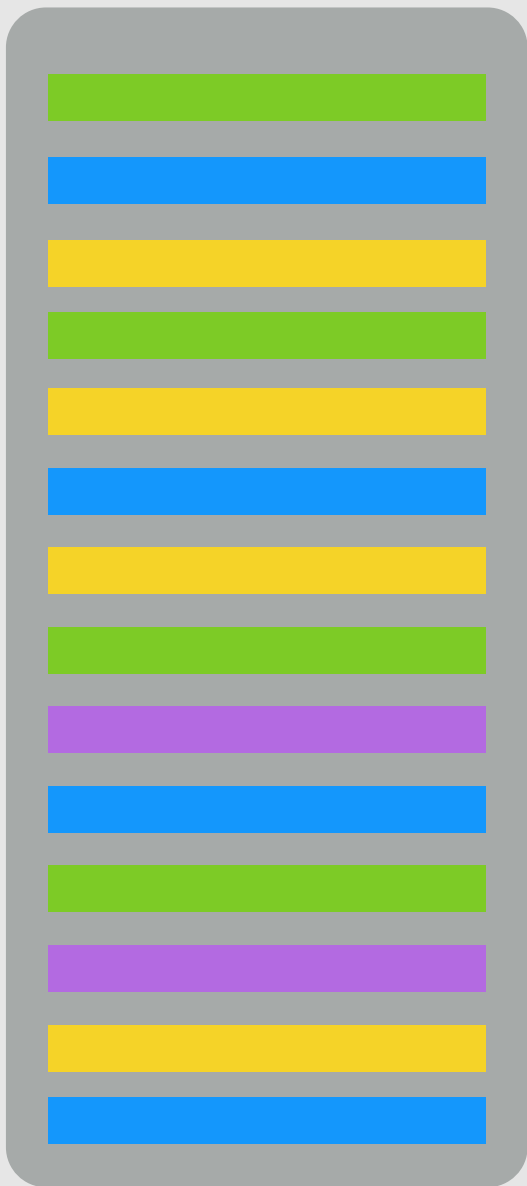
Revenue from Product



Capital Redeployment

Remaining

Product Backlog



Revenue from
Product Backlog



Actual Cost



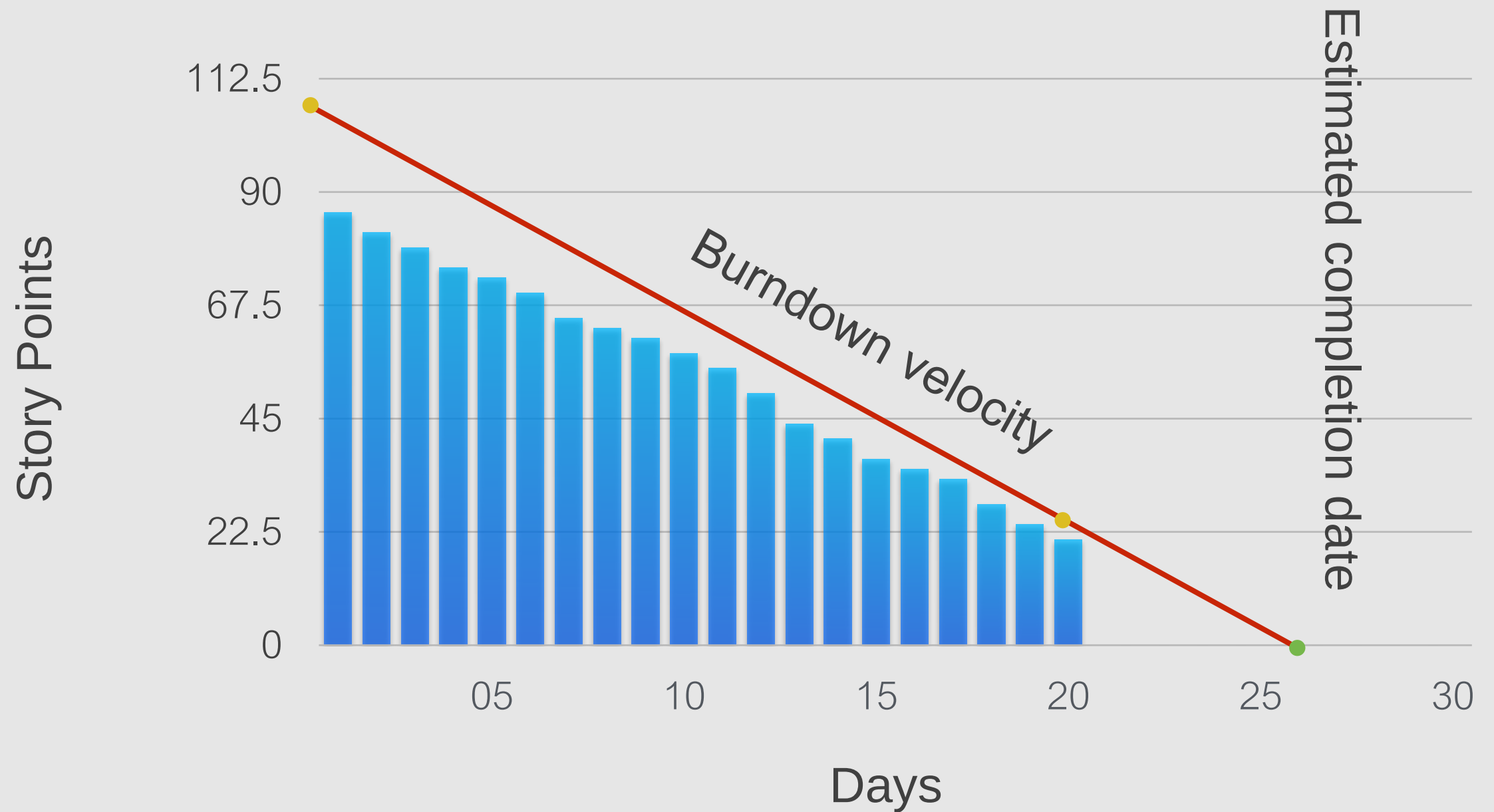
Scrum Metrics

- Daily scrum and sprint retrospective
- Team satisfaction
- Team member turnover

Scrum Reporting

- Sprint and release turndown - view of progress at a glance
- Sprint velocity - review of value delivered over time
- Scope change - stories added during the release
- Team capacity - FT developers? Vacations? Sick leave? Other projects?
- Escaped defects - how is your production software

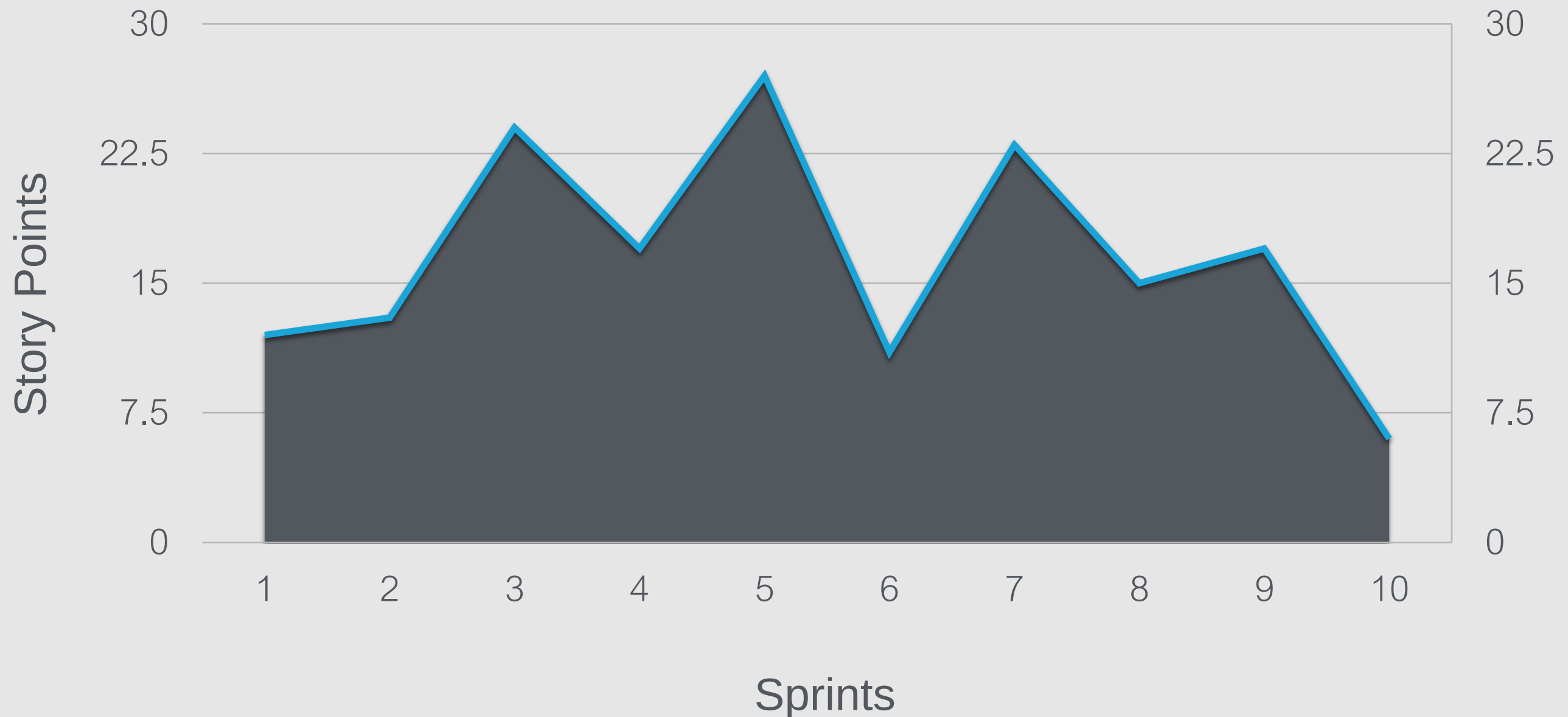
Sprint Burn Down



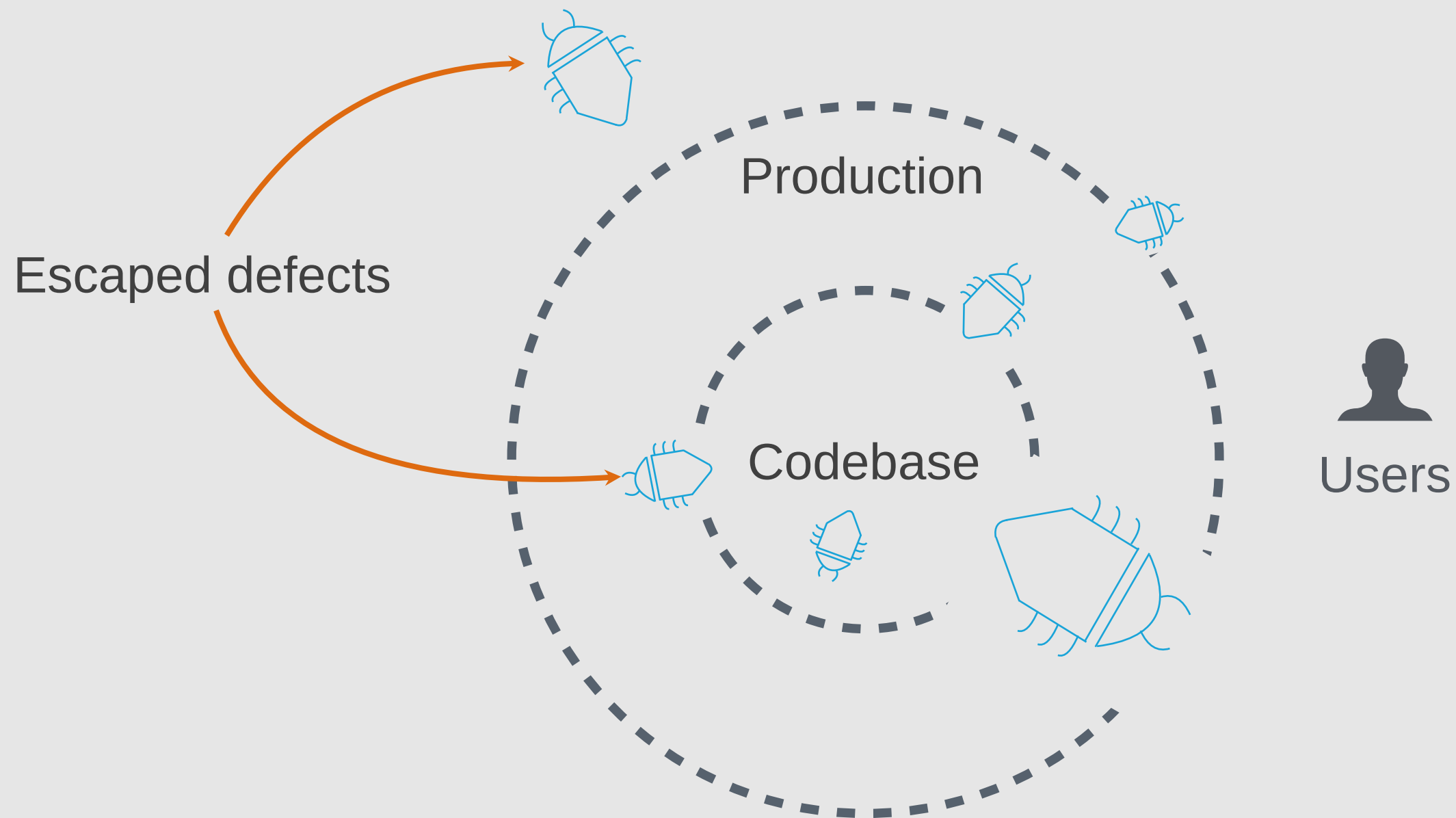
Team Velocity



Don't use as a goal
or to compare teams



Escaped Defects

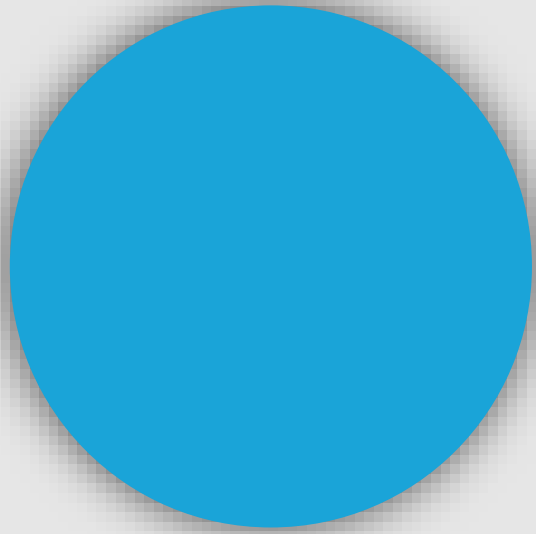


Harmful Metrics

Flawed Metrics

- Lines of code
- Developer ranking
- Commit frequency
- Pull request count
- Velocity or story points
- Impact

Lines of Code



Complexity of Problem



Complexity of Solution

LOC



Lines of Code

- Advantages
 - A measure of the systems size
 - proxy for complexity or resources
 - Helps calculate defect density
 - Measuring can be automated (cheap)
 - Correlates with cost

Lines of Code

- Disadvantages
 - Vanity metric - not linked to progress
 - Easy to game
 - Accounts for only 30-35% of overall effort
 - Increases code complexity
 - Poor measure of an individuals productivity
 - doesn't account for developer experience
 - Poor measure of system functionality
 - Language dependent
 - Doesn't account for automated code
 - No standard
 - Comments count? are data declarations in? lengthy statements?

Lines of Code Example



```
float InvSqrt(float x)
{
    float xhalf = 0.5f * x;
    int i = *(int*)&x;
    i = 0x5f3759df - (i >> 1);
    x = *(float*)&i;
    x = x*(1.5f - xhalf*x*x);
    return x;
}
```



```
// store floating-point bits in integer
// initial guess for Newton's method
// convert new bits into float
// One round of Newton's method
```

Strong Metrics

- Cycle time - Speed
- Mean time between failures (mttr) - Responsiveness
- Change failure rate (defect density) - Quality
- Deployment frequency - Throughput

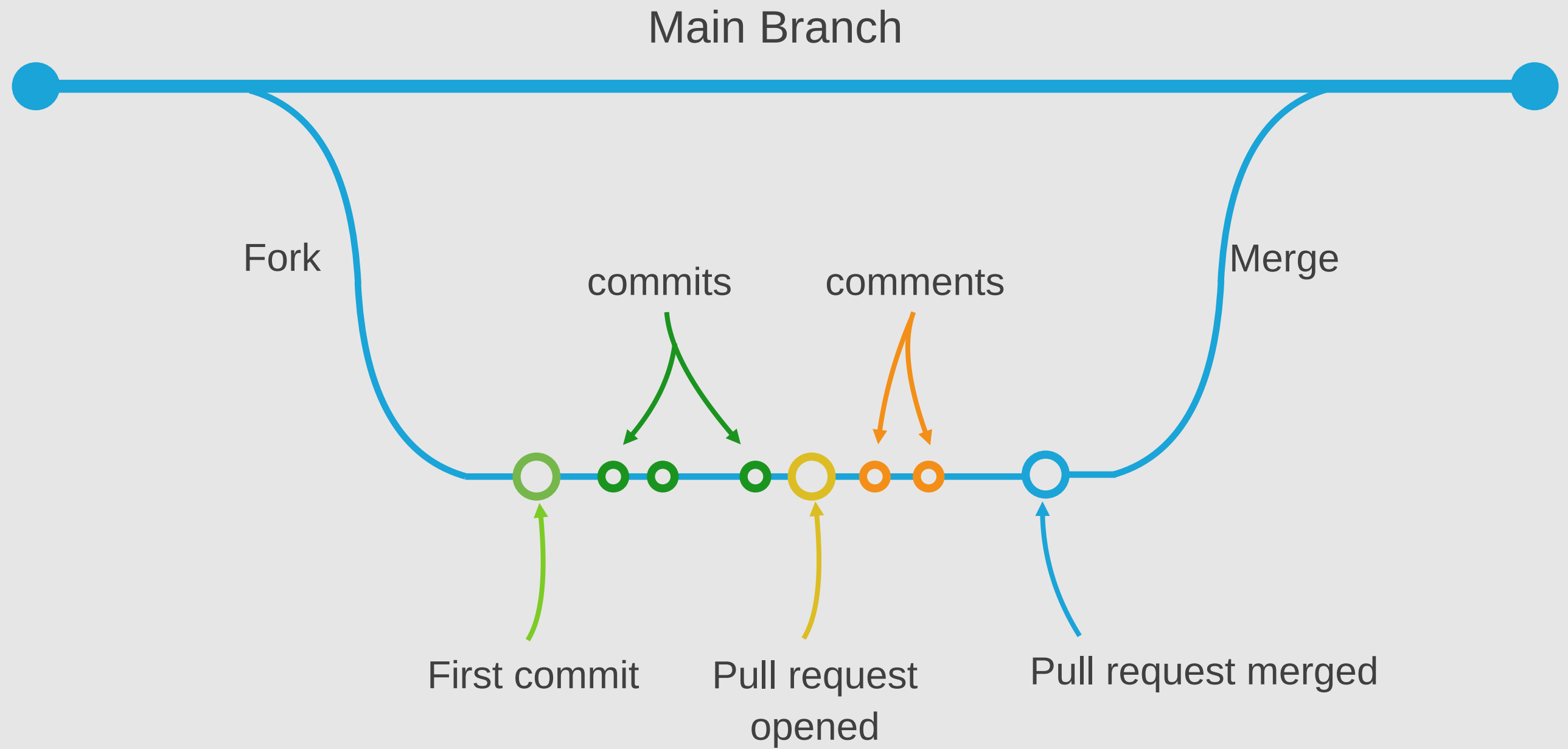
GIT Analytics

GIT Analytics

- On track to deliver
- Delivery speed
- Identify bottlenecks
- Productivity
- Code review

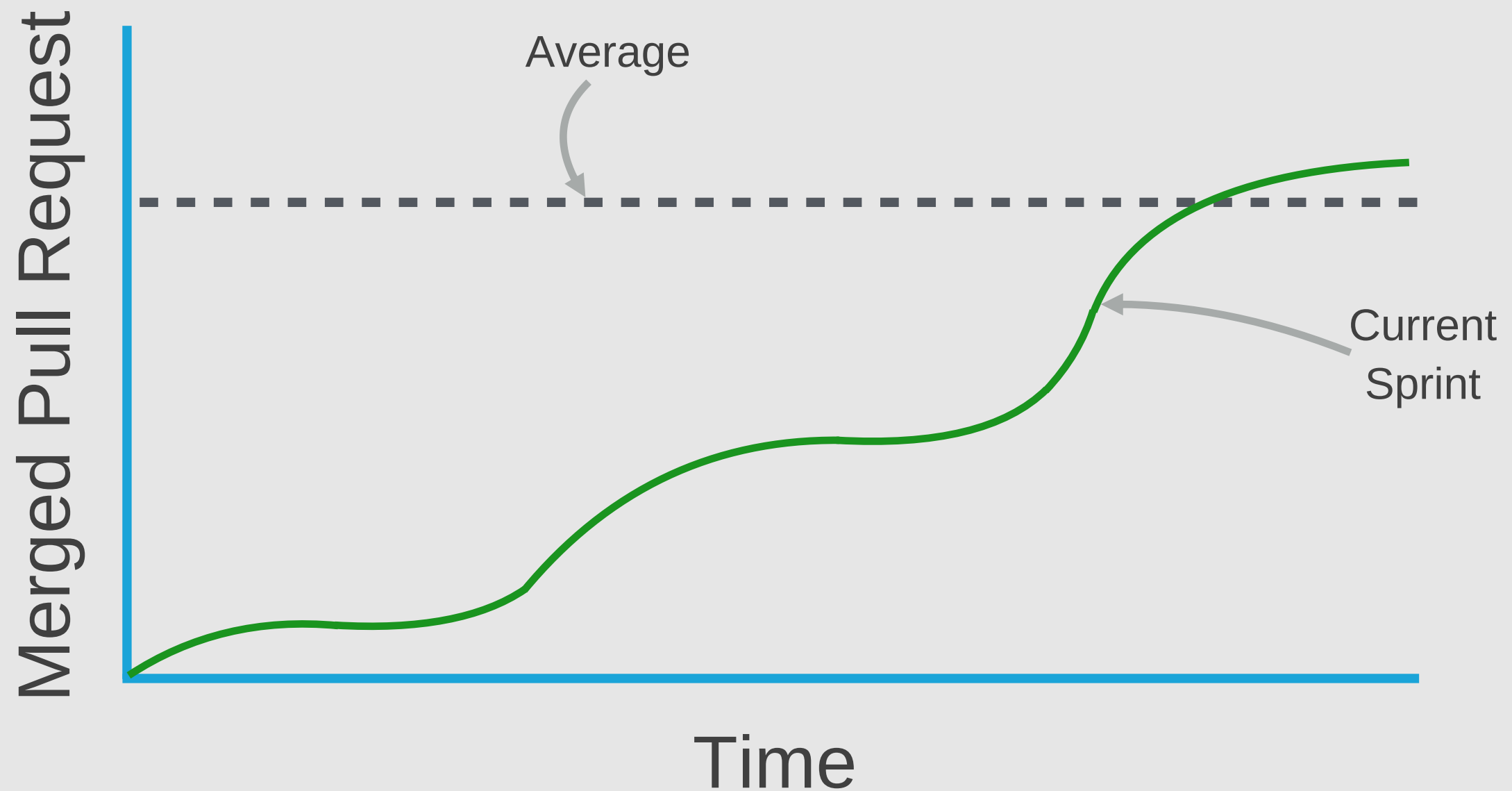
On Track to Deliver

Merged Pull Request



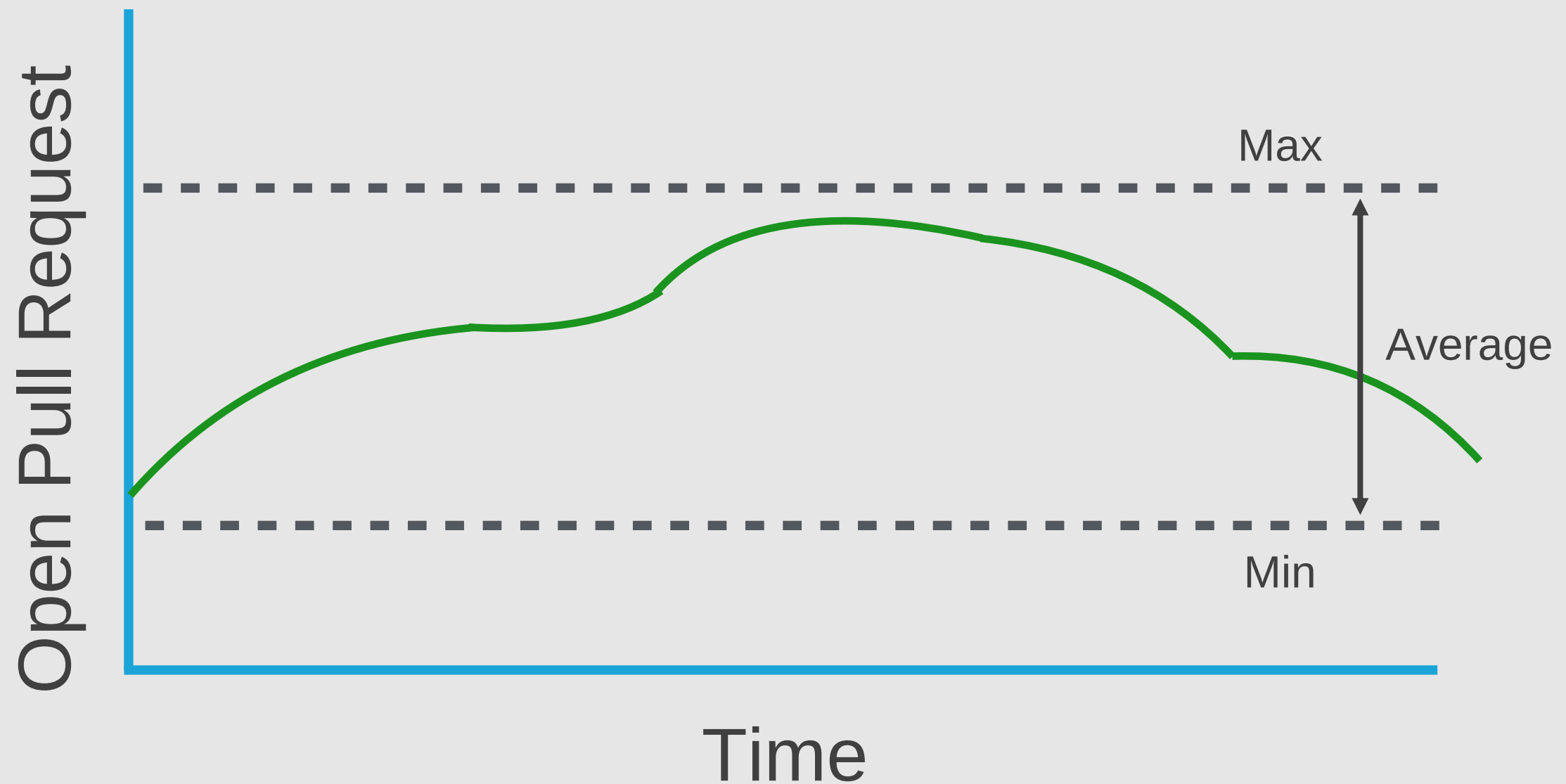
Merged Pull Request in Sprint

Steady rise in number of merged pull requests towards team average

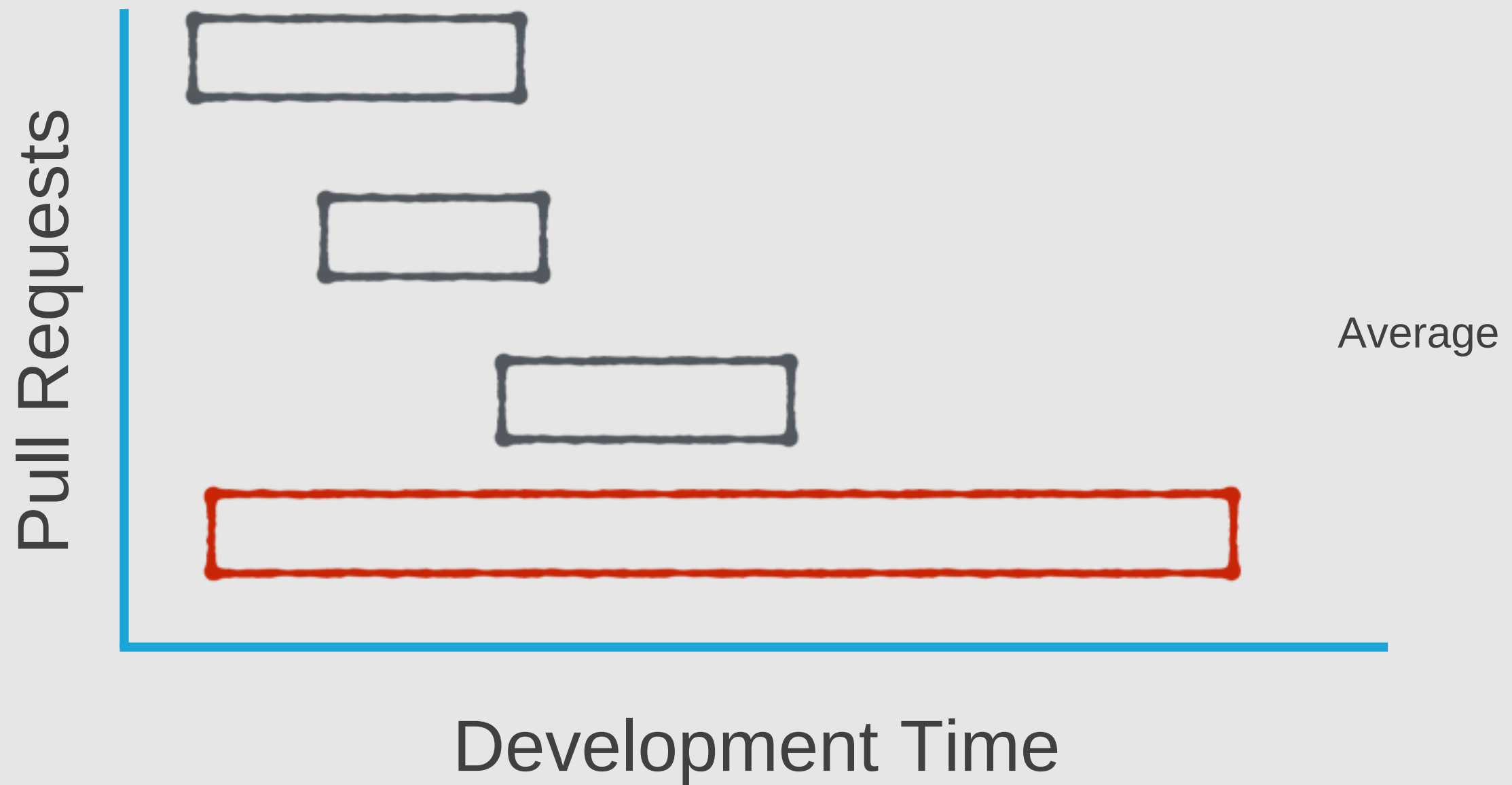


Opened Pull Requests

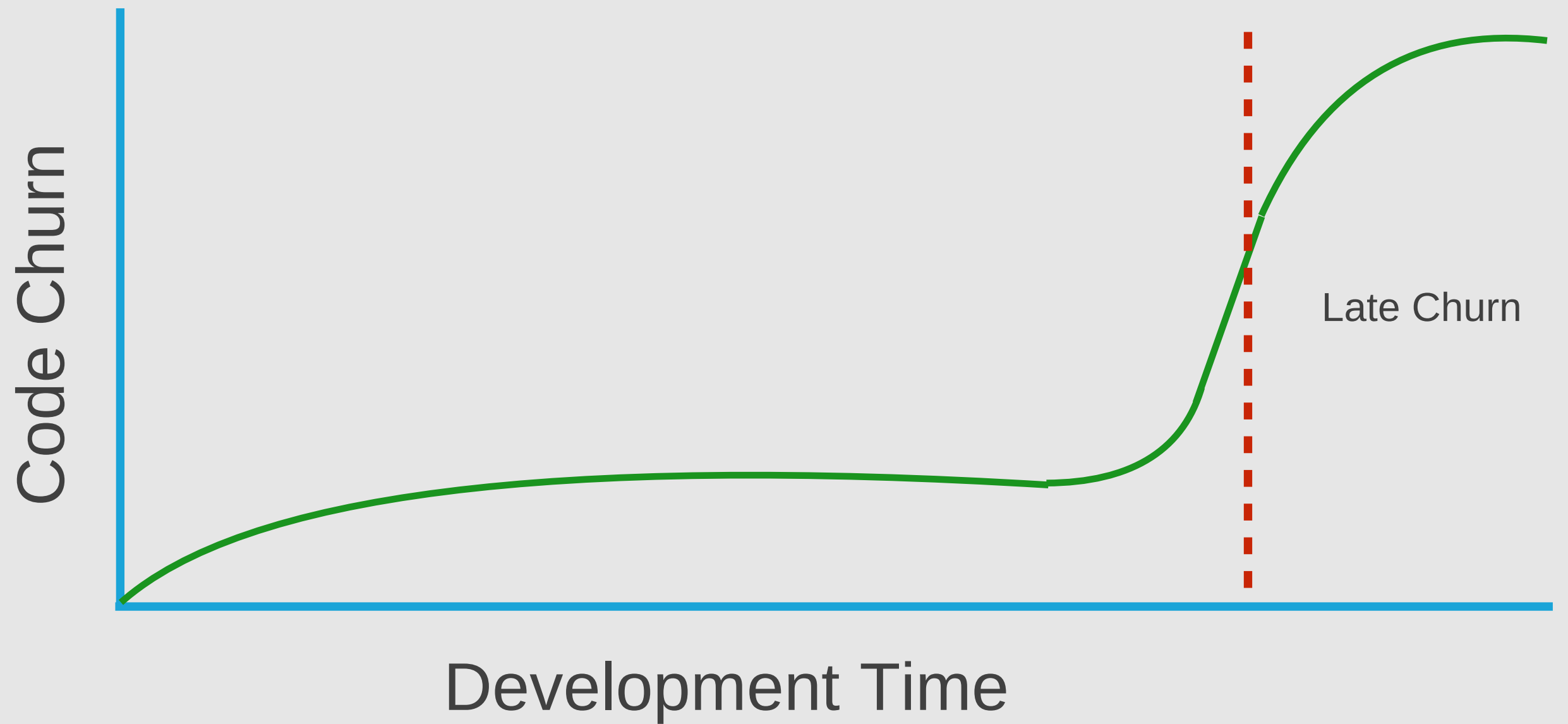
Steady rise in number of open pull requests near middle of the sprint team keeping with max and min limits is all healthy



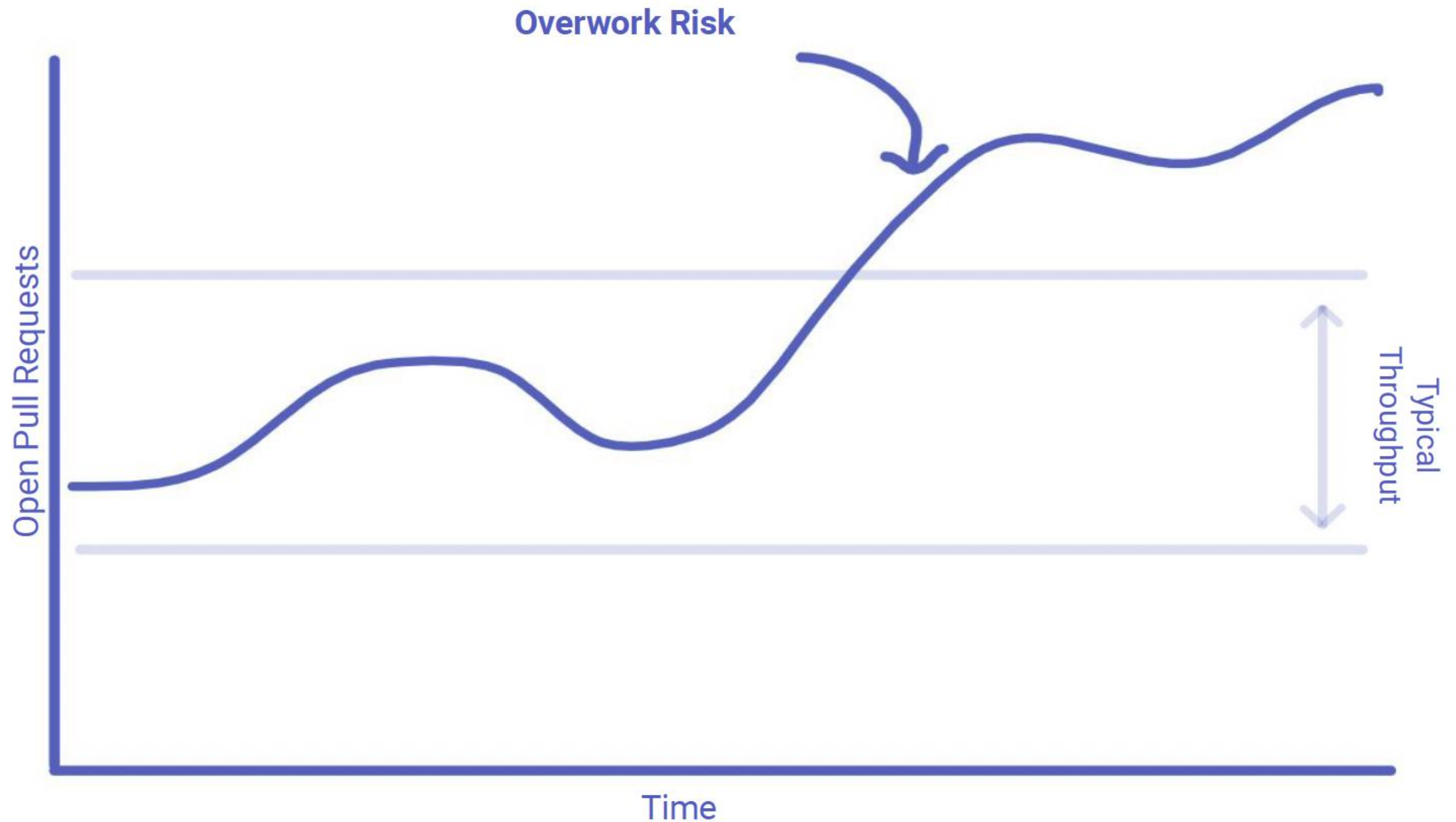
Long time in development



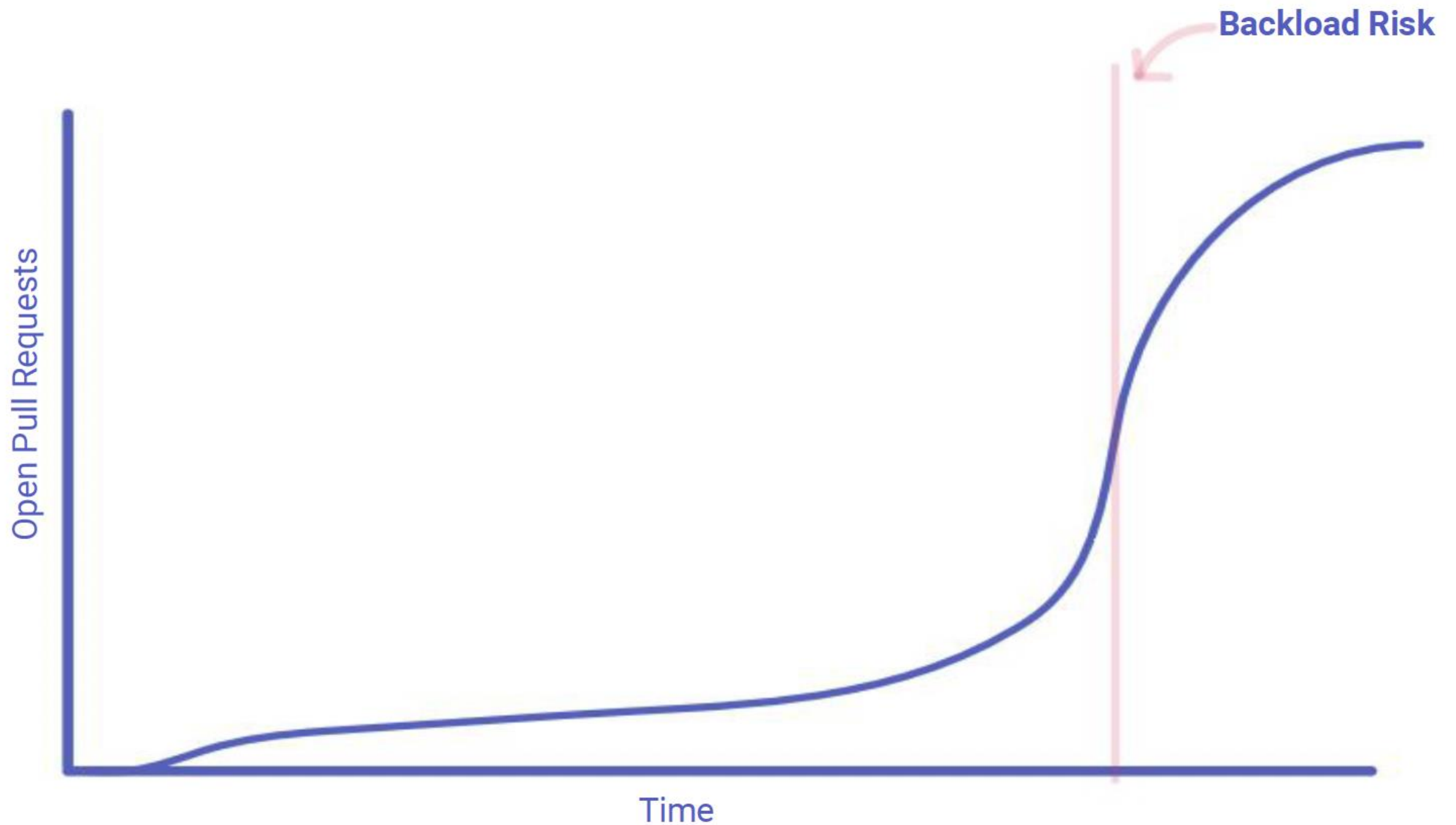
Late Churn



Overload Risk

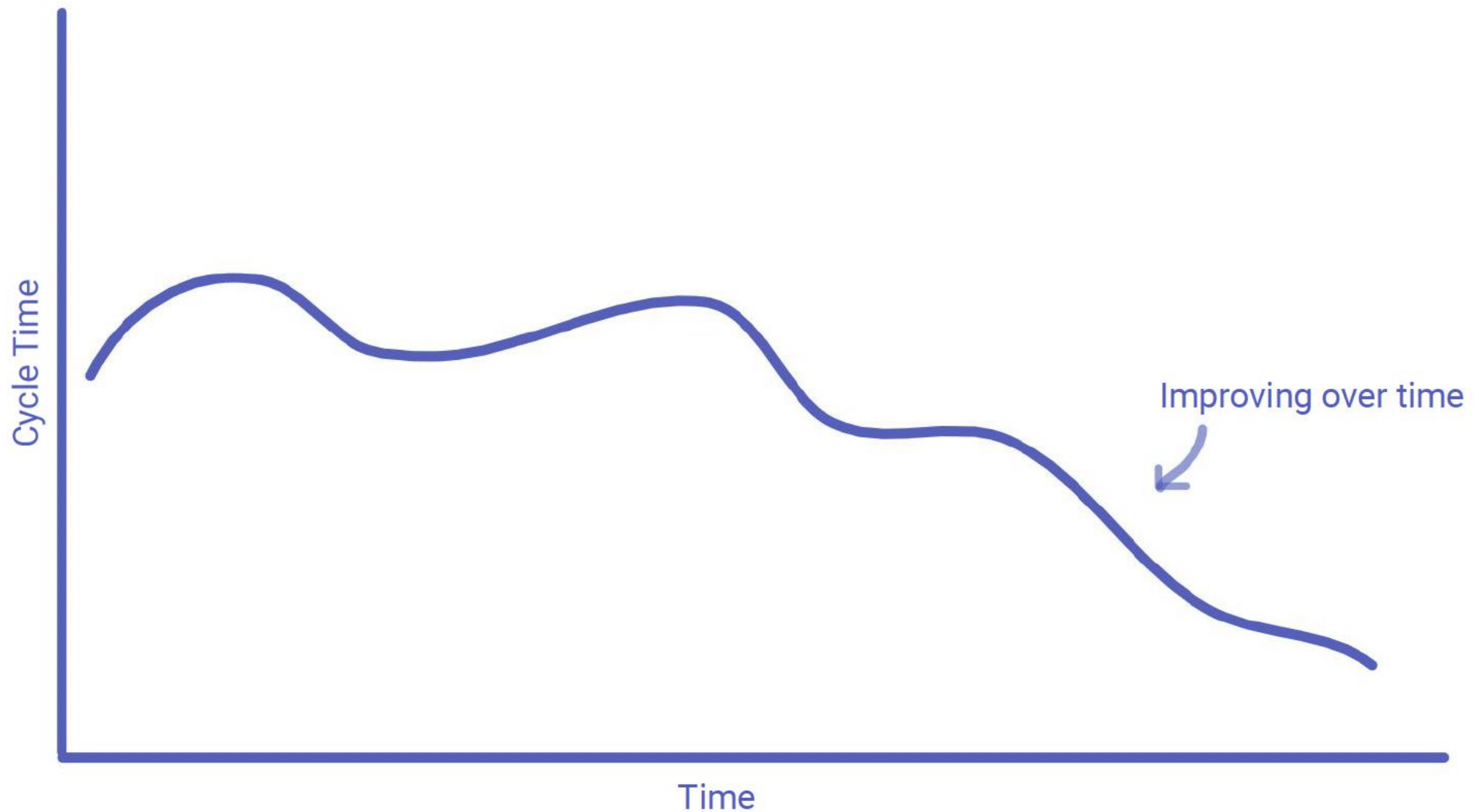


Backload Risk

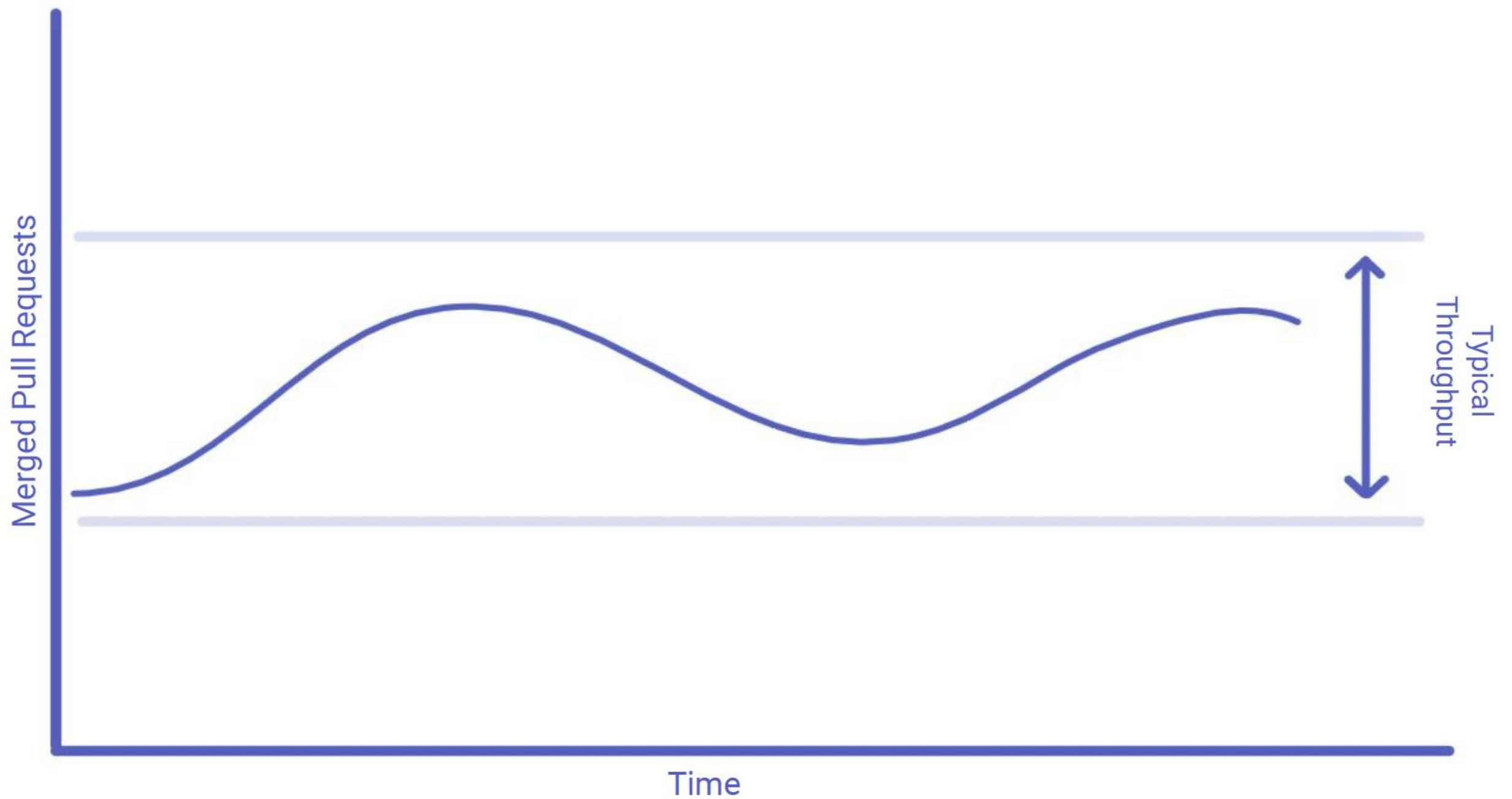


How we Deliver

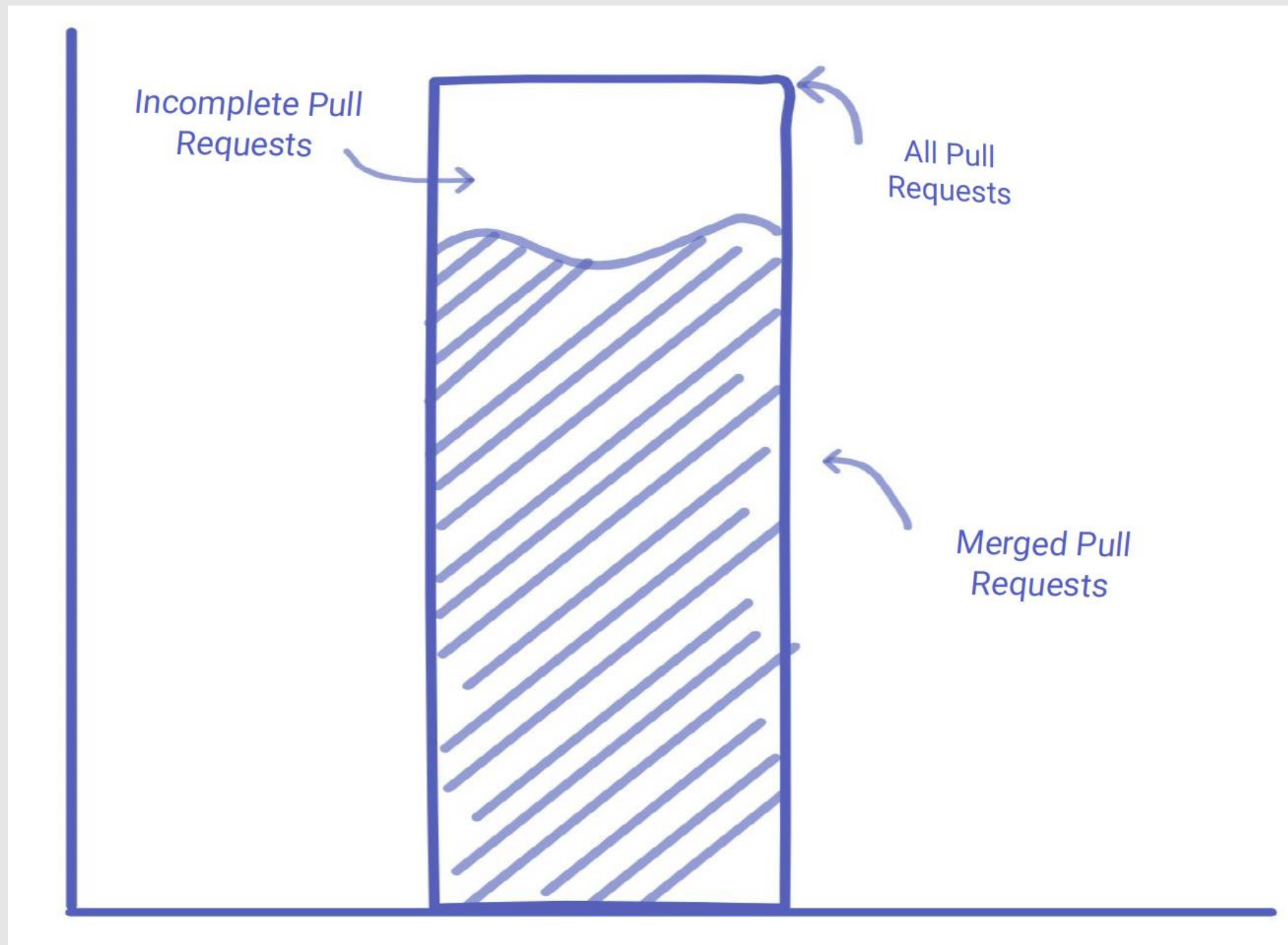
Cycle Time - aka Speed



Throughput



Merge Rate

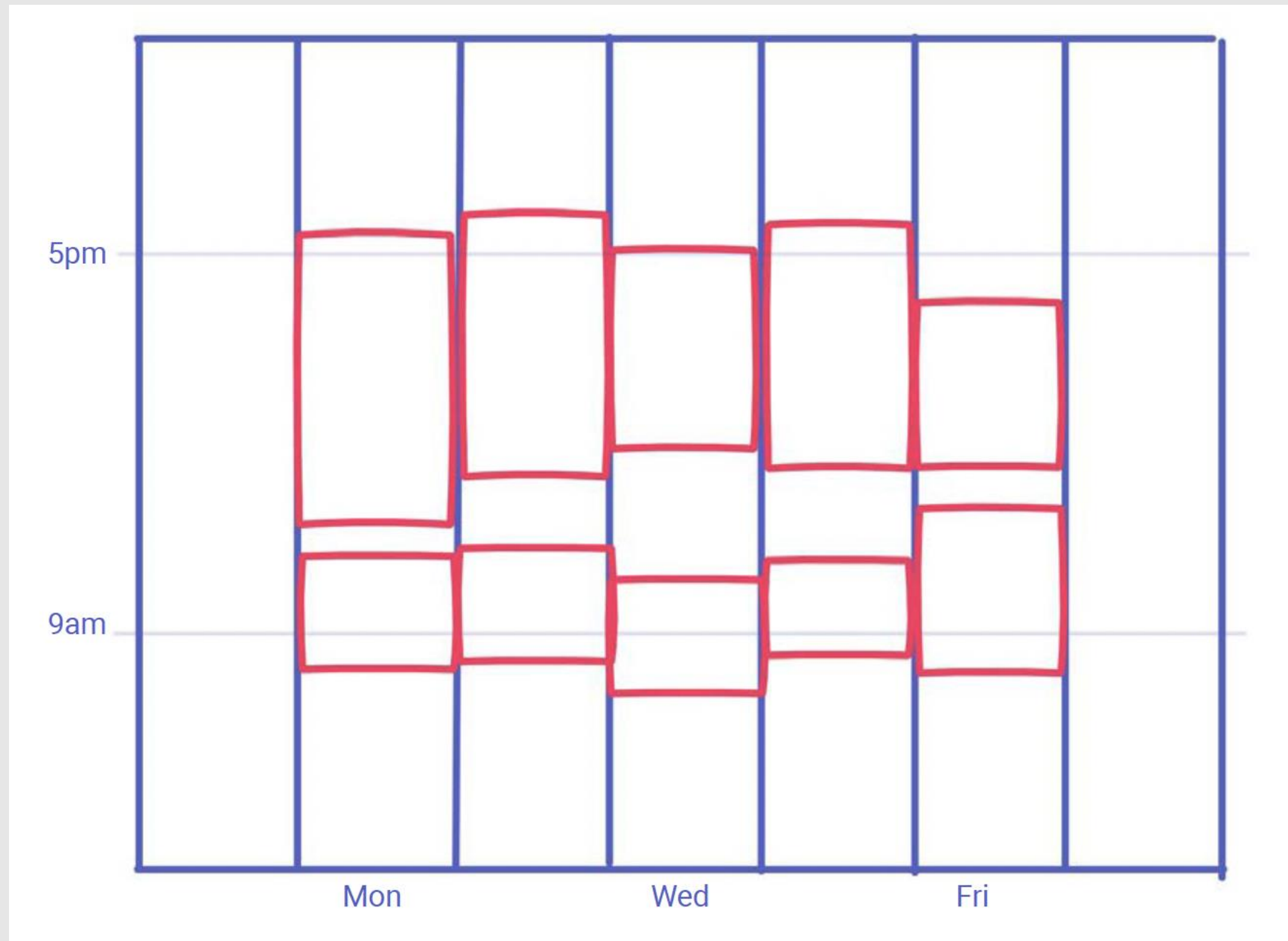


Surfacing Inefficiencies

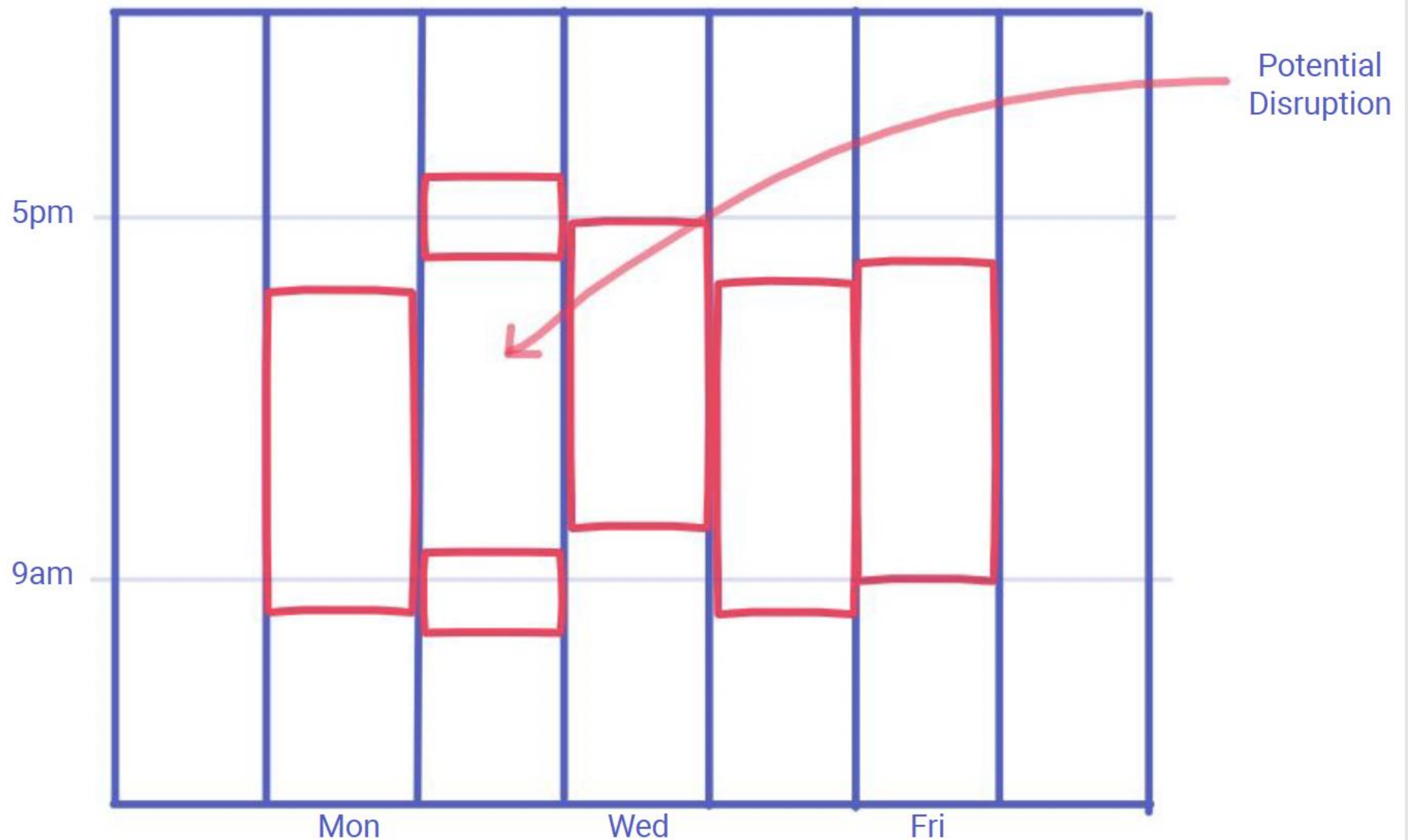


When are we Productive

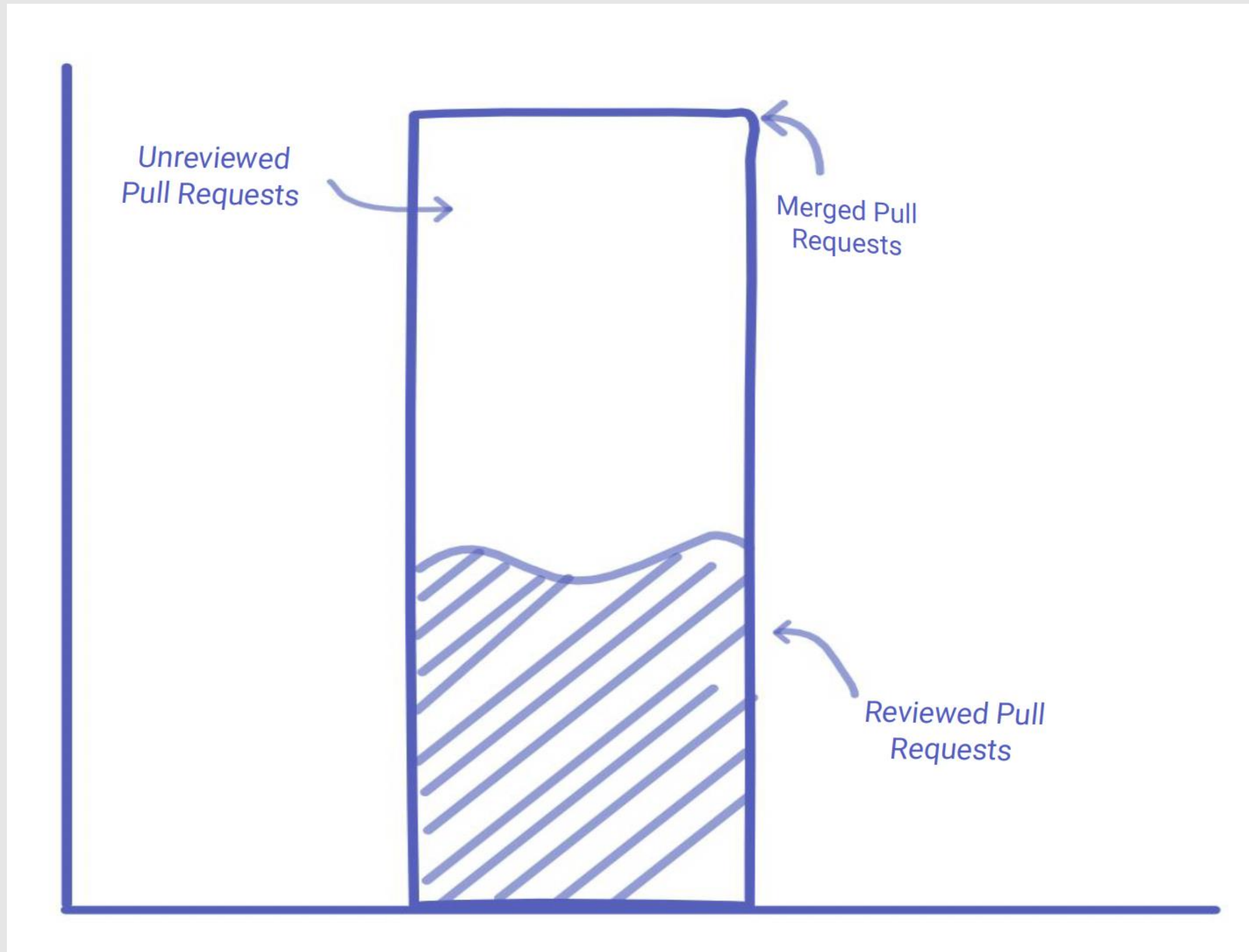
Activity Heatmap



Protecting Deep Work

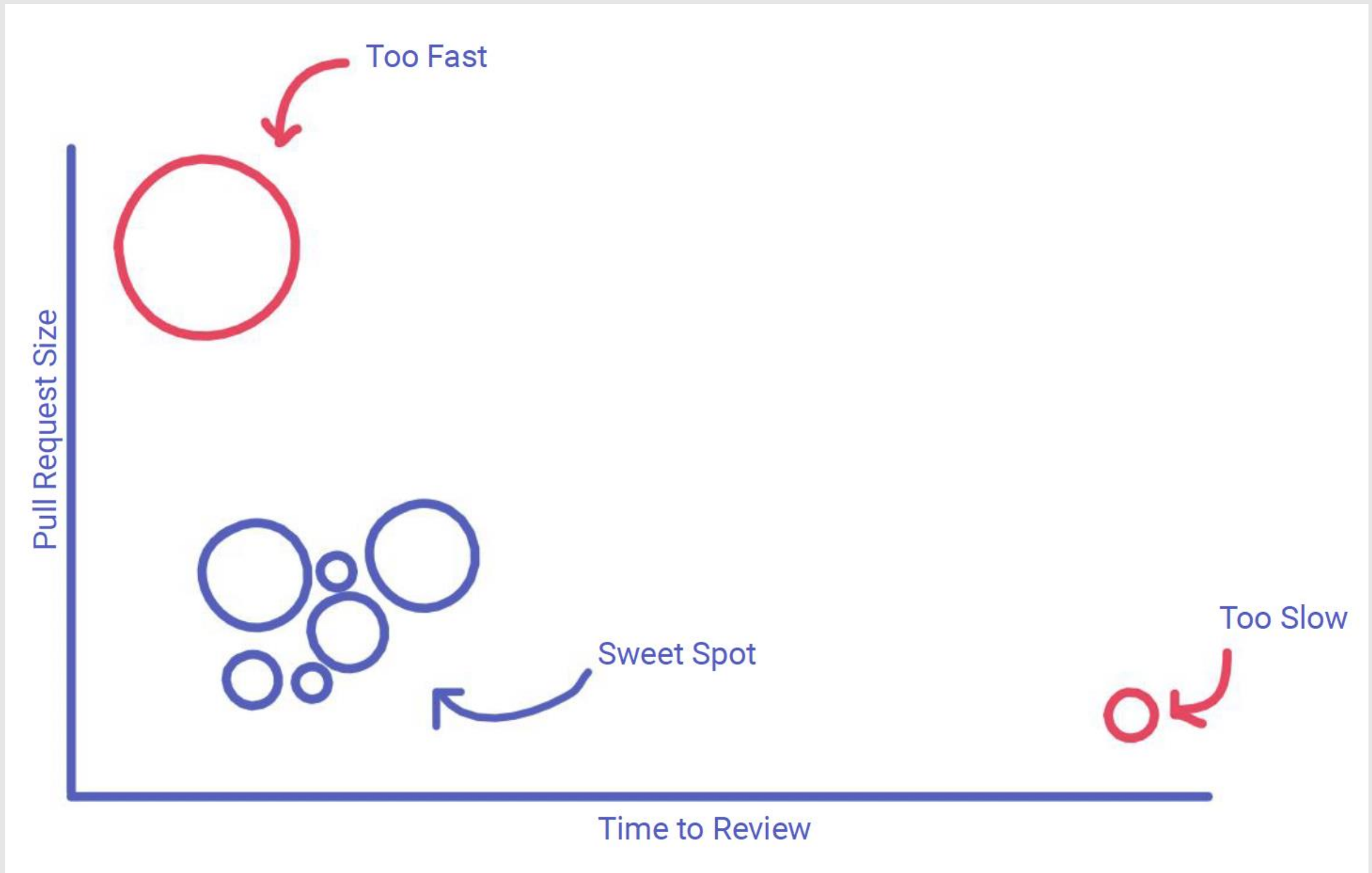


Moving too fast

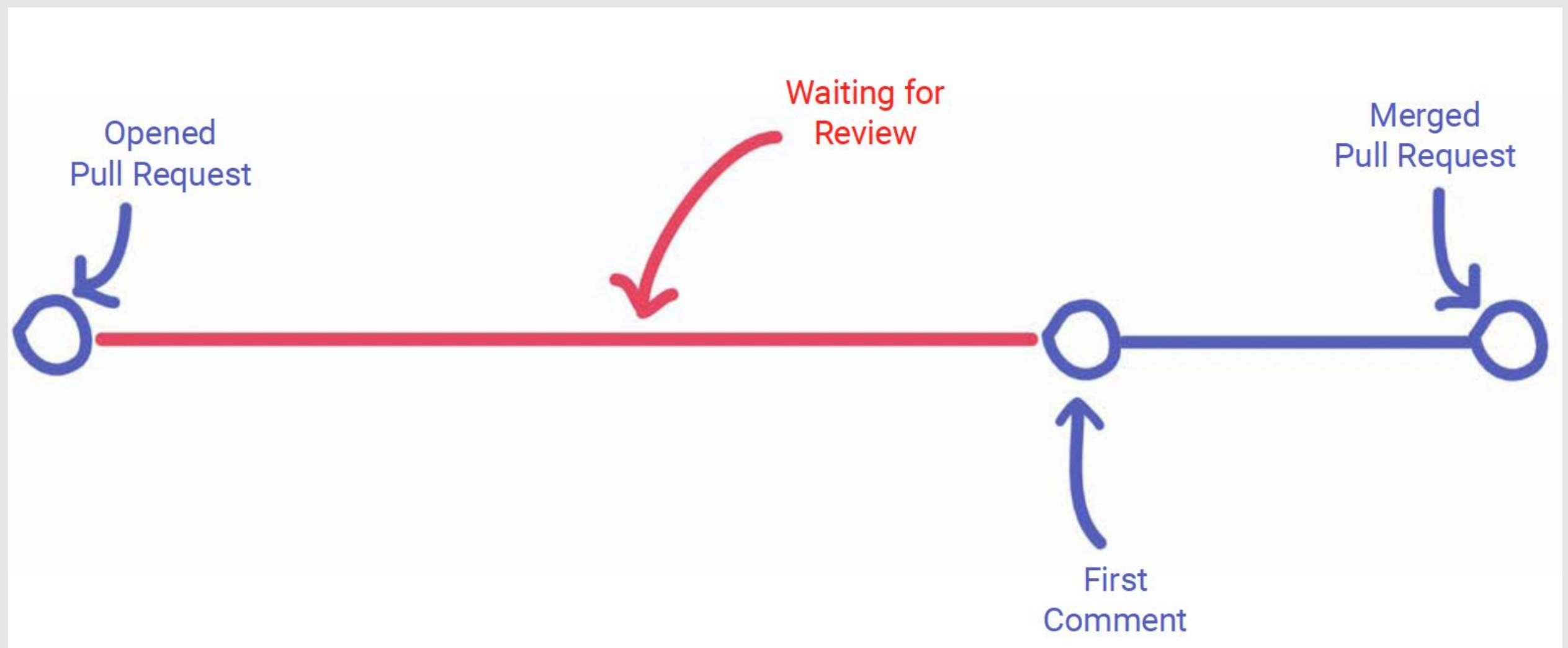


How do we do in code review

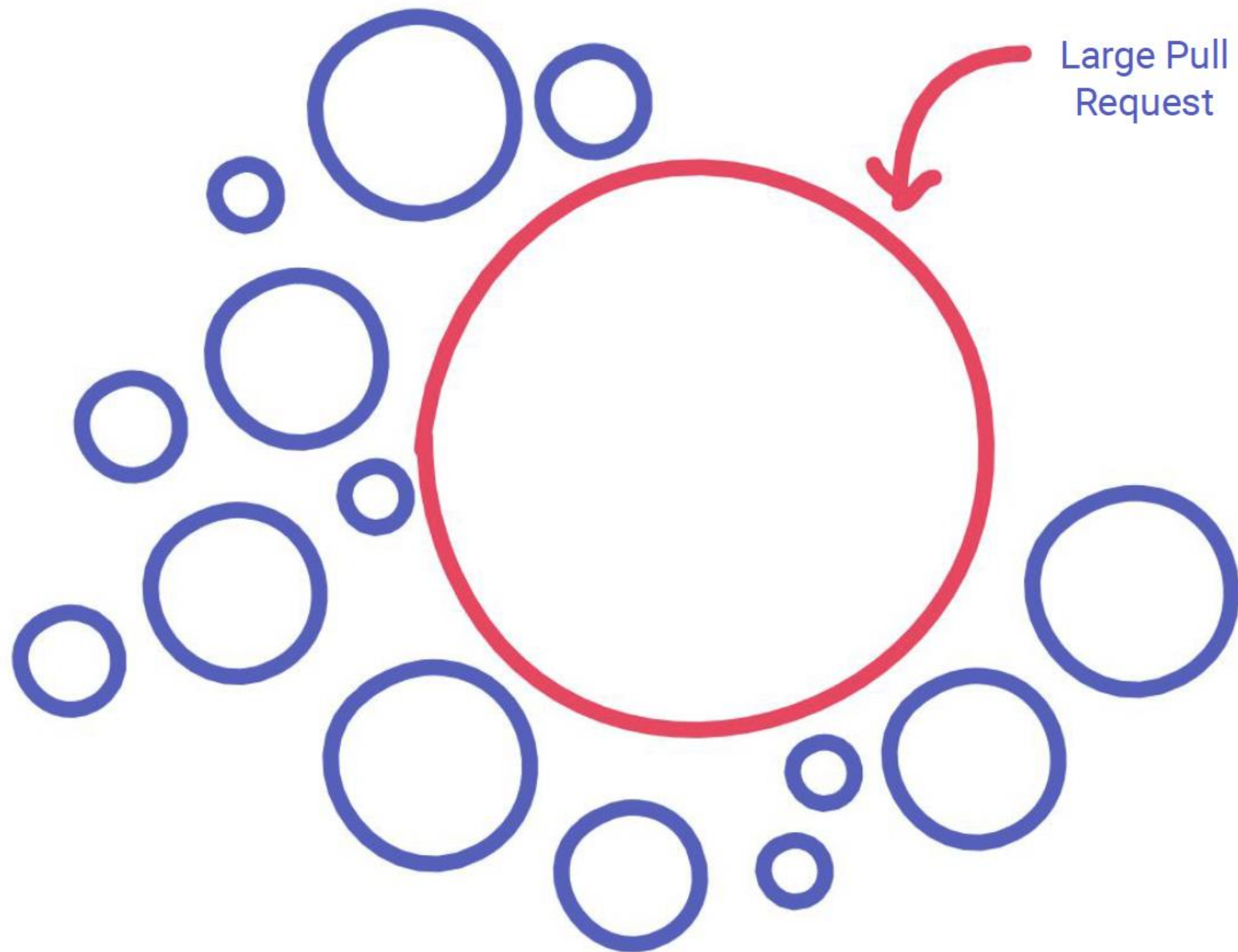
Size vs. Time to Review (moving too fast)



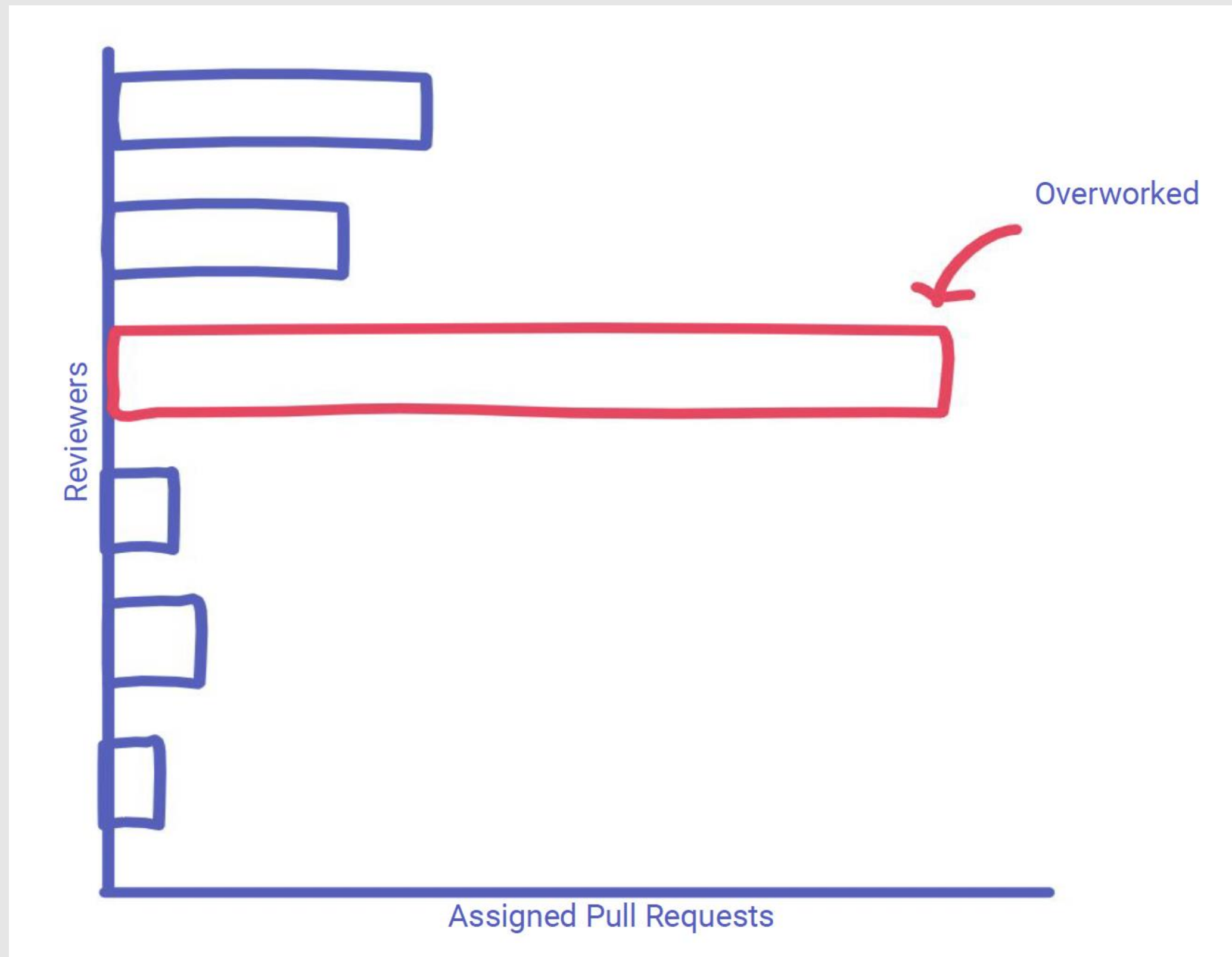
Response Time



Pull Request Size

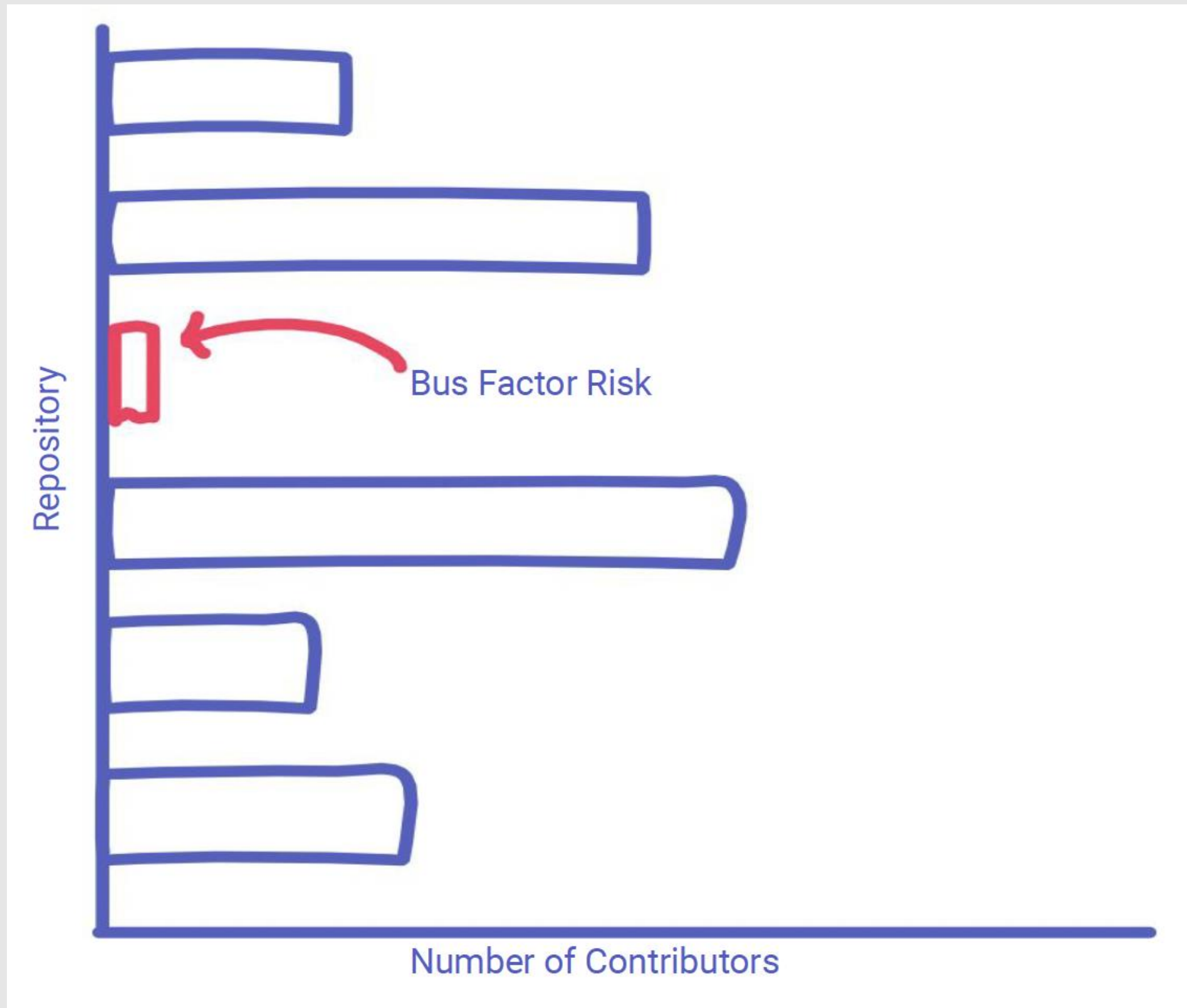


Review Distrubtion



How do we share knowledge

Spotting Bus Factor



Encouraging Communication - Number of Reviewers

